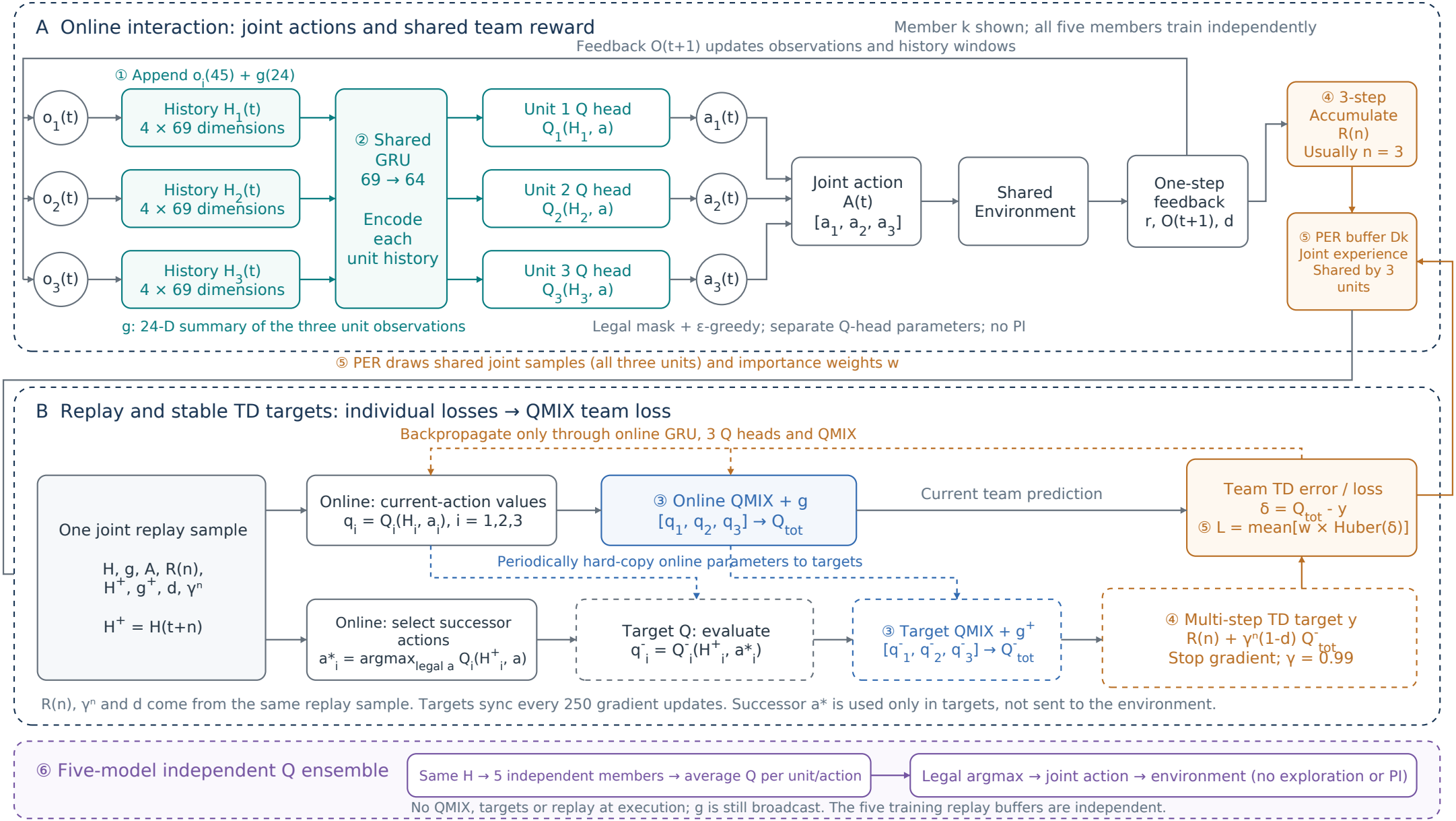
Based on original MADQN

New M4: Improvements to the MADQN Architecture

Retain joint interaction and Double DQN; add global features, GRU, QMIX, 3-step, PER and five-model independent Q ensembling.

Legend: black/gray = retained MADQN; colors 1-6 = added/modified; small dashed boxes = targets or target computation.



Network Components and Training Roles

Online networks, target networks and ensemble members have distinct roles in M4.

Controller overview

Upgrade three workshop supervisors into a team: read a plant-wide dashboard, remember recent changes, and learn from plant-wide performance. During training, look several steps ahead and revisit difficult experiences. At deployment, five independently trained teams score each action.

Diagram component	Meaning in this project
3 agents	Control CONWIP caps for A processing, B processing and assembly.
5 independent models	Five training runs of the same algorithm, not five production units.
Online / target networks	A learner and its delayed copy within one model, not ensemble members.
PI OFF	No PI advice input, residual decoding or PI-guided exploration.

Key differences from the MATD3 reference

MATD3 has actors, twin critics and a minimum over two target values. Here, discrete-action value learning scores three candidate actions, and QMIX learns a team value.

There are no actors, twin-critic minimum or target-action noise. Double-DQN-style targets use online action selection and target-network evaluation.

Two different meanings of centralization

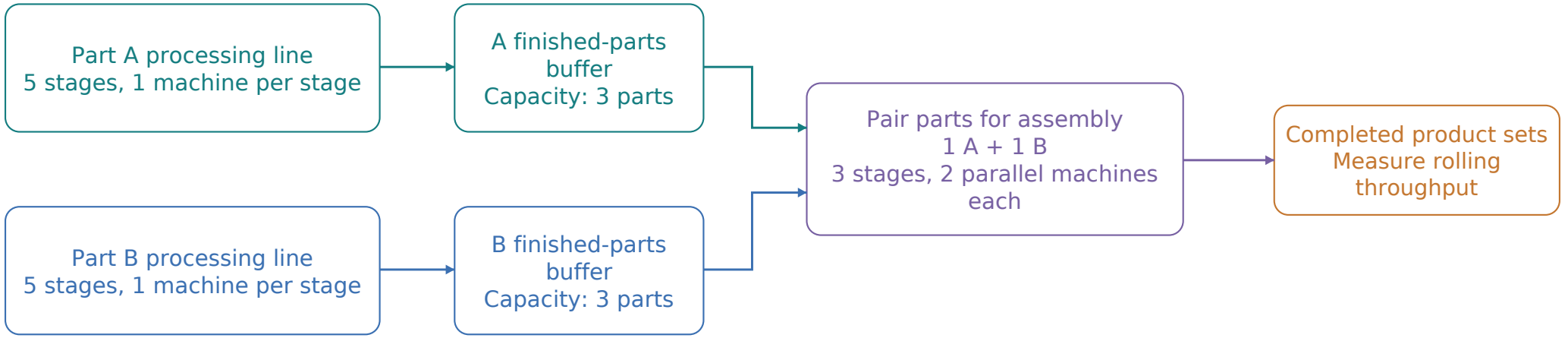
QMIX is used only during training, but the 24-D global summary is also broadcast during execution. Execution is therefore not strictly local and communication-free.

Suggested reading order

Pages 3-4: problem and objective. Pages 5-13: individual improvements. Pages 14-16: learning and control end to end. Pages 17-19: what the experiments establish. Page 20: glossary and evidence.

Production: Two Part Lines Must Deliver a Matched Pair

The controller changes permitted work-in-process in each unit, not machine speed.



Like making burgers: plenty of buns cannot replace missing patties.

Where does the difficulty come from?

A/B processing times vary randomly, throughput targets change, and individual stages temporarily slow down. Releasing more WIP can feed a starved downstream unit or fill an already crowded buffer. A busier local line does not guarantee a better plant.

There is no separate random breakdown/repair module. Disturbances come from processing-time randomness, target changes and processing-mean changes.

Simulation time scale

Warm up for 720 minutes, then control for 3600 minutes. Decisions occur every 10 minutes: 360 per episode. Throughput and reference use 120-minute rolling windows. Initial caps: A=3, B=3, assembly=2.

Six consecutive 600-minute regimes

Targets: 3 → 6 → 5 → 5 → 5 → 3 sets/hour. Regime 3 increases A/B variability; regime 4 slows B stage 3 by 15%; regime 5 instead slows A stage 2 by 15%; the last restores normal conditions.

Good Control Is More Than Minimizing Throughput Error

Tracking, inventory and cap adjustments share one reward.

Throughput tracking error
Larger deviation from the rolling target costs more

+

Work-in-process inventory
More work tied up in production costs more

+

Cap-adjustment magnitude
Larger and more frequent changes cost more

$$r(t) = -\left[\text{tracking cost} + 0.05 \times \text{mean WIP} / 38 + 0.005 \times \Sigma|\text{cap change}| / 3\right]$$

Why are rewards usually negative?

Reward is negative total cost, so −35 is better than −39. Reward ×1000 is the episode's mean per-step reward multiplied by 1000, then averaged across test episodes. It is neither cumulative return nor exploratory training reward.

Tracking cost is squared normalized error. Episode NRMSE is the square root of mean tracking cost and excludes inventory and adjustment costs.

From original MADQN to new M4

The matched MADQN baseline uses three feedforward Q networks, current 45-D observations, independent learning, 1-step targets and uniform replay. New M4 adds spatial coordination, short-term memory, team-value learning and training enhancements, then ensembles five models.

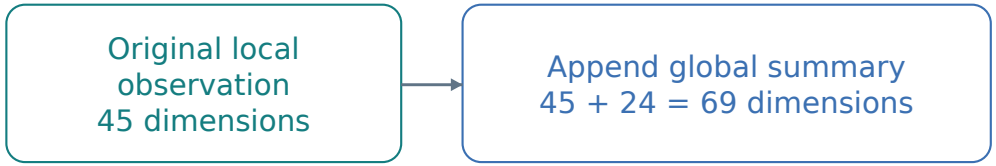
The contribution is environment-specific integration and validation. GRU, QMIX and PER themselves are established methods, not inventions of this project.

WIP accounting: A WIP + B WIP + 2 × assembly WIP + both finished-parts buffers. Each assembly set contains two parts.

Global Features: A Plant-Wide Dashboard for Every Supervisor

Address the blind spot where one line looks healthy but the bottleneck is elsewhere.

The 24-D summary	Contents
6-D: unit states	WIP and cap for A, B and assembly.
6-D: shared production information	A/B buffers, throughput/reference, relative error, current target and window target.
12-D: operating summaries	Busy, blocked, idle and pair-starvation summaries for each unit.



These features summarize three observable local vectors; they do not reveal the future. The original 45-D observation already includes shared buffers and throughput, so removing global features does not eliminate every plant-wide signal.

Why does this help this production system?

Suppose A's buffer is almost full, B's almost empty, and assembly waits for a pair. An A supervisor focused only on local utilization may release more work and worsen accumulation. Seeing B and assembly makes plant-wide balancing possible.

Characteristics, benefits and costs

The summary links the three units spatially so Q networks can learn who is waiting for whom. It requires timely broadcast during execution. The ablation removes only these 24 dimensions from Q-network inputs; QMIX retains its 24-D conditioning input.

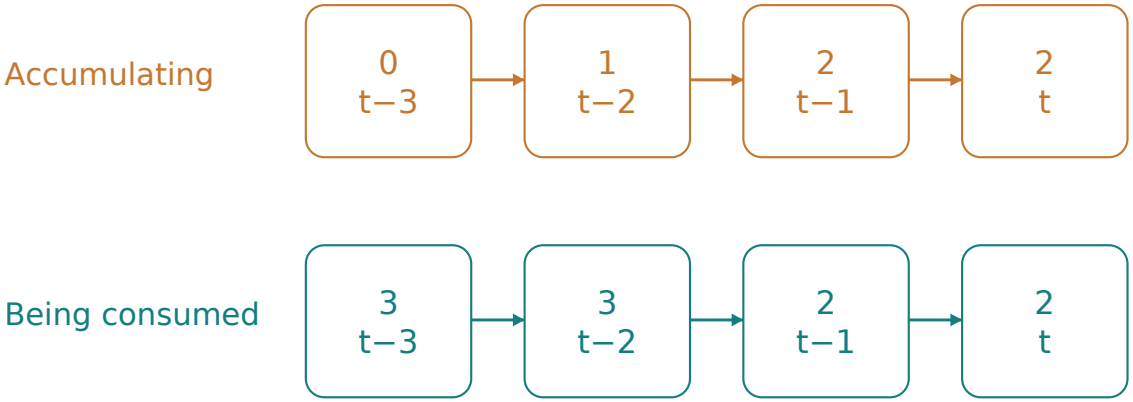
Ablation evidence: retaining this module improves reward

Without module, Reward ×1000: -38.602; full version: -35.540.Mean advantage of retaining the module: 3.061. Conditional on other modules and five-model ensembling; not its standalone benefit.

GRU: Watch a Short Video, Not Just a Photograph

GRU stands for Gated Recurrent Unit.

The same buffer count of 2 can describe two different situations



The final frame is identical but the trends differ. A current-frame network cannot easily identify the path taken. GRU compresses recent frames into short-term memory, giving action scoring information about direction of change.

Illustration only. Actual actions also depend on the target, pair starvation and plant-wide state. These counts alone do not determine an action.

Gates do not retain all past information equally

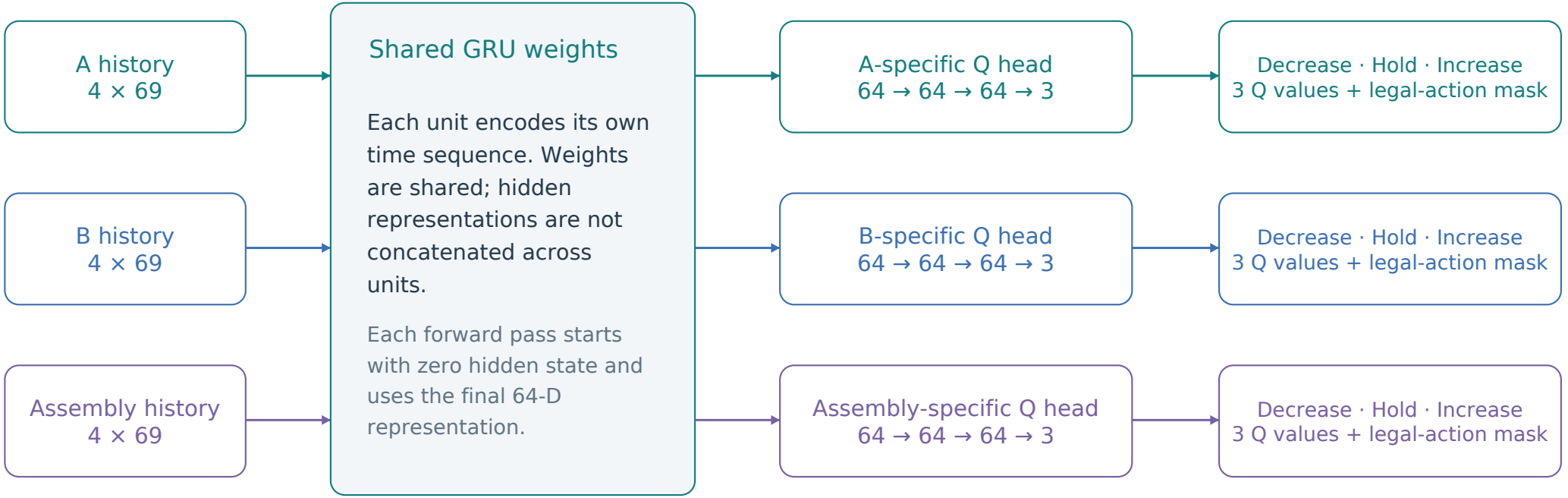
The update gate controls how much old memory remains. The reset gate controls how much old information contributes to a candidate memory. Noise can be downweighted while persistent blocking or target changes are retained. Gate values are learned, not hand-coded thresholds.

Improvement over original MADQN

The original feedforward network scores the current observation directly. GRU first interprets a short sequence. The same error may mean a new deviation or an ongoing recovery. Memory can distinguish these and reduce reactions to momentary fluctuations.

GRU in Code: Shared Encoder, Distinct Roles

Each forward pass reads the latest 4 frames; hidden state does not persist across the whole episode.



What is shared, and what is independent?

Within one model, GRU weights are shared but the three output heads have separate parameters to preserve unit roles. Across five models, GRUs, heads, QMIX, optimizers and replay buffers are independent.

Four frames cover t−3 through t, spanning 30 minutes. At episode start, repeat the initial frame until the history is full.

Ablation evidence: retaining this module improves reward

Without module, Reward ×1000: -39.358; full version: -35.540.Mean advantage of retaining the module: 3.818. Conditional on other modules and five-model ensembling; not its standalone benefit.

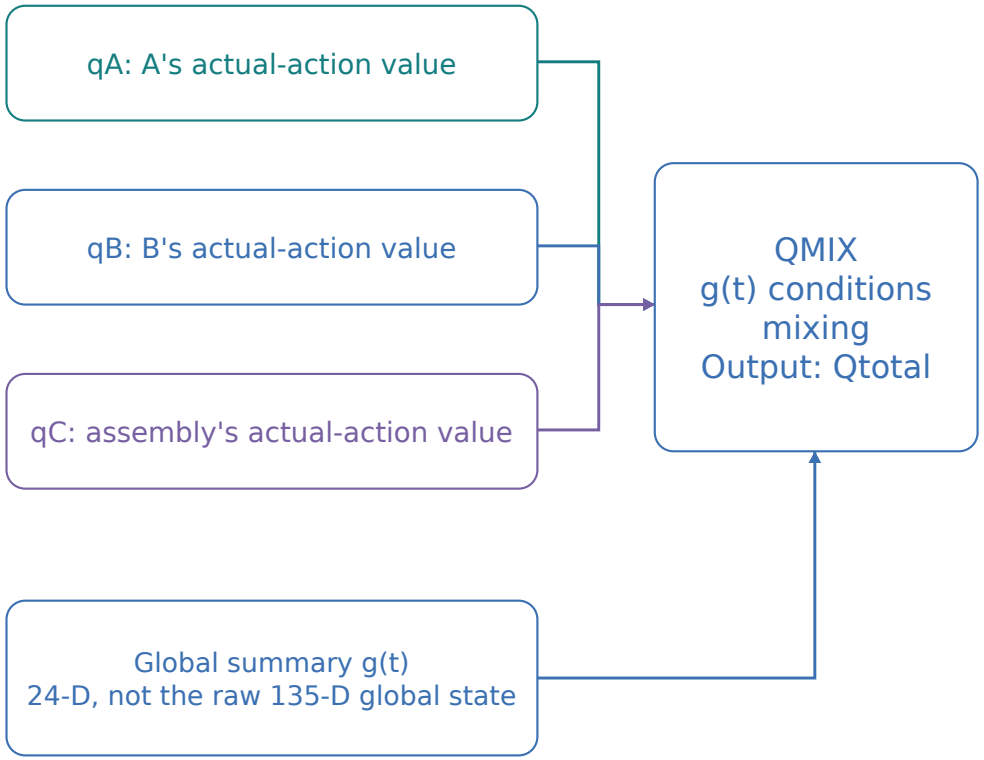
QMIX: Combine Three Local Scores into a Team Score

A shared reward reports plant performance; QMIX also learns how the agents' values combine.

The difficulty with independent learning

If completed-product throughput falls, A, B and assembly all receive the same poor reward. Should B's cap rise or A's accumulation shrink? Independent updates from a shared reward make it harder to connect an agent's action with its teammates.

QMIX combines the three actual-action Q values, conditioned on the global summary, into team value Qtotal.



Like a coach reviewing a relay race

The coach examines handoffs, not just individual running speed. Mixer loss backpropagates through all three Q heads and the shared GRU, aligning local action scores with the shared objective.

QMIX does not explicitly calculate contribution percentages or manually assign responsibility.

Local Q values are internal team scores

They need not represent physical throughput or separate unit profits. The main training constraint acts on the mixed team value. Do not interpret the three Q values as true contribution shares.

QMIX Monotonicity: Why Local High Scores Matter

For a fixed state, increasing any local Q cannot decrease the mixed team Q.

$$Q_{total} = f(q_A, q_B, q_C ; g), \text{ with } \partial Q_{total} / \partial q_i \geq 0$$

Implementation: a 24-D global summary generates two mixing layers' weights. Absolute values enforce nonnegative weights; the hidden layer uses ELU and width 32.

What does the constraint enable?

Imagine three knobs connected to one dashboard. Turning any knob toward a higher local score cannot lower the team score. With fixed inputs and separable legal-action constraints, independent local argmax choices maximize the mixed value.

Training can evaluate the team while execution avoids enumerating all joint actions through QMIX. See [1].

Describe this project's execution precisely

Deployment does not run the mixer, but each Q network still reads the broadcast global summary. Averaging local Q values across models adds another operation. Single-model monotonicity does not guarantee that the ensemble action maximizes the mean of five Qtotal values. Ensemble benefits are empirical.

Not a universal team model

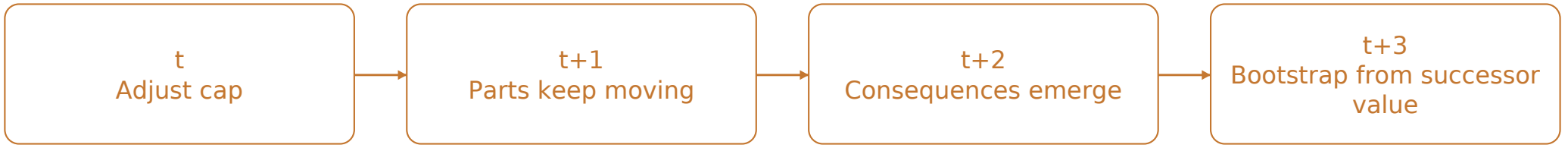
Monotonicity limits expressiveness. Arbitrary nonmonotonic coordination relationships, where a locally preferred action must be sacrificed, cannot always be represented exactly.

Ablation evidence: retaining this module improves reward

Without module, Reward ×1000: -38.432; full version: -35.540.Mean advantage of retaining the module: 2.891. Conditional on other modules and five-model ensembling; not its standalone benefit.

3-Step: Do Not Judge a Release Before Its Effects Arrive

n-step does not peek into the future. It waits for actual transitions, then learns from several consequences together.



$$y = r(t) + \gamma\, r(t+1) + \gamma^2\, r(t+2) + \gamma^3\, (1-d)\, Qtotal-(t+3)$$

Usually n = 3. Near episode end use the actual remaining n, with no bootstrap after termination. γ = 0.99.

Why look several steps ahead in production?

After a cap increase, extra parts are still in transit and throughput may not rise immediately. A one-step target may initially see only inventory cost. Three-step returns bring rewards from three actual decision intervals back to the current action.

Three steps cover about 30 minutes, not every processing or rolling-window delay. Longer effects still rely on value bootstrapping.

Numerical illustration (not experimental data)

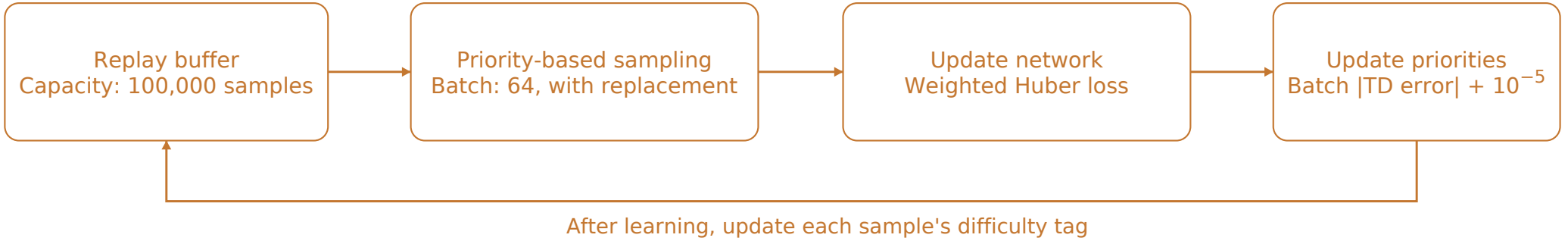
For rewards −0.04, −0.03 and −0.02, the discounted short return is approximately −0.0893, plus 0.99^3 times the successor value. This extends the learning horizon rather than tripling reward. Longer returns can increase variance and off-policy bias; larger n is not always better.

Ablation evidence: retaining this module improves reward

Without module, Reward ×1000: -38.036; full version: -35.540.Mean advantage of retaining the module: 2.496. Conditional on other modules and five-model ensembling; not its standalone benefit.

PER: An Updating Notebook of Prediction Mistakes

Prioritized Experience Replay assigns priority according to prediction error.



Difficult does not mean lowest reward

A bad outcome can have low TD error if already predicted accurately. An unexpectedly good outcome can have high TD error. PER favors experiences the current model cannot yet explain, not only failures.

Target switches or congestion turning points may therefore receive more replay. The code does not hand-assign priority to regime changes.

How does PER avoid focusing only on mistakes?

Sampling probability is proportional to priority raised to 0.6, moderating concentration. Importance weights reduce nonuniform-sampling bias; β rises from 0.4 to 1 and weights are batch-normalized. New experiences receive the current maximum priority.
See [2].

Different from n-step

n-step sets how far one sample's target looks ahead. PER sets how often that sample is revisited. They address different issues.

Ablation evidence: retaining this module improves reward

Without module, Reward ×1000: -37.311; full version: -35.540.Mean advantage of retaining the module: 1.771. Conditional on other modules and five-model ensembling; not its standalone benefit.

Five-Model Independent Q Ensemble: Average Scores, Then Decide

Not majority voting, action averaging, or averaging five separate rewards and calling that deployed performance.

Five evaluators, each fully trained

The five members share an architecture but independently initialize, explore and learn under different seeds. Each has its own replay buffer, optimizer, targets and best validation checkpoint.

At execution, all receive the same history. Average Q arithmetically for each unit and candidate action, then select the highest-scoring legal action.

$$\text{Mean } qi(a) = [q^1i(a) + \ldots + q^5i(a)] / 5$$

$$ai = \text{argmax of mean } qi \text{ over legal actions}$$

Observed change in this test

New M4 single-model mean: -39.486; five-model joint deployment: -35.540. Mean NRMSE falls from 0.13171 to 0.12302. These compare actual control trajectories.

Why is it not voting?

Illustrative member	Decrease Q	Hold Q	Increase Q
1	−1.0	−1.2	−2.0
2	−1.0	−1.2	−2.0
3	−1.0	−1.2	−2.0
4	−4.0	−0.8	−2.0
5	−4.0	−0.8	−2.0
Mean	−2.20	−1.04	−2.00

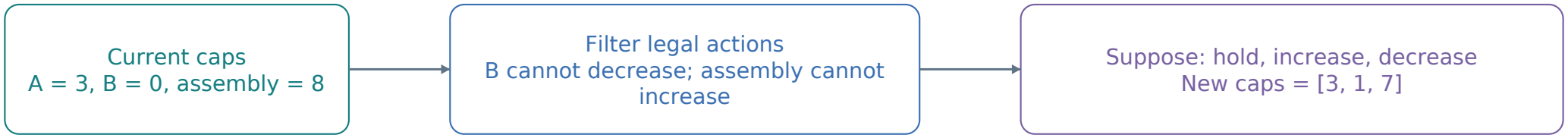
Teaching numbers, not model outputs. Voting chooses decrease by 3:2. Mean-Q selection chooses hold because −1.04 is highest. Score margins also affect the decision.

Benefit: may reduce accidental single-run bias. Cost: roughly five networks' training and inference. Q scales are not explicitly calibrated, and errors are not guaranteed to cancel.

With PI Off, Networks Directly Choose Cap Changes

M4 remains the project version name; this does not imply retaining the old PI branch.

Aspect	Earlier PI-residual variant	Current version: PI OFF
Input	69-D features + 3-D PI action encoding.	69-D; no PI advice appended.
Output meaning	Correction around the PI proposal.	Direct decrease, hold or increase.
Exploration / initialization	May use PI guidance and a zero-residual prior.	No PI guidance or PI-specific hold bias.
Boundary protection	Check legality after residual mapping.	Mask illegal actions using current caps directly.



Action indices and their meaning

Indices 0 / 1 / 2 mean cap changes −1 / 0 / +1. Caps stay within 0-8 and illegal candidates are masked. Reducing a cap does not remove existing WIP; it limits subsequent releases.

One fewer module and dependency

The new version needs no PI formula, advice or residual constraint. The choice follows existing project results, not a claim that PI is always useless. All following ablations keep PI off.

Learning from One Batch: Replay to Parameter Update

Expand the training part of the overview and identify what each arrow carries.

1

Sample a batch

PER draws 64 experiences: current history, global summary, actual actions, multi-step return, successor history and terminal flag.

2

Compute current estimate

Online GRU and Q heads score actions. Gather executed-action Q values and combine them with online QMIX into current Qtotal.

3

Compute a stable target

At the successor history, online Q selects the best legal action. Target Q evaluates that action, then target QMIX produces successor value.

4

Backpropagate

Multi-step reward plus discounted successor value gives y. Weighted Huber loss between Qtotal and y updates online parameters.

5

Maintain the learning system

Update replay priorities using this batch's |TD error|. Every 250 gradient updates, copy online weights to targets.

Two networks, two responsibilities

Online network: selects the best successor action and receives gradients for the current estimate.

Target network: provides a steadier estimate of that selected action. Target y is detached.

Separating selection from evaluation helps reduce overestimation. See [3].

Do not draw this as TD3

This is online selection and target evaluation, not the minimum of twin critics. The five members do not average each other's TD targets; they train independently.

Training, Validation and Test Have Different Jobs

This guide uses completed experiments. No retraining or new seeds; comparison definitions remain consistent.



Validation seeds: 16101-16112. Test seeds: 30101-30130. Validation selects models; test compares them. Do not interchange the two.

Training setting	Current implementation
Optimizer / learning rate	Adam / 0.0001
Learning trigger	After ≥1000 replay samples, update every 4 interactions
Batch / gradient clipping	64 / norm limit 10
Discount / return length	0.99 / 3 (shorter at episode end)
Exploration ε	1 → 0.05 over the first 108,000 steps
Target synchronization	Hard copy every 250 gradient updates

How is the final model selected?

Training seeds: 13101-13105. Validate on 12 fixed seeds every 50 episodes. Reward gain >0.05 (×1000 units) is an improvement. If reward differs by at most 0.05, an NRMSE decrease >0.0002 can update the best point.

Train at least 300 episodes; stop after 4 validations without improvement, with a 1250-episode limit. Use the best validation checkpoint, not the final episode.

Early stopping is not complete mathematical convergence

Across six configurations and 30 members, 27 meet validation early stopping and 3 reach the budget limit. New M4, no-GRU and no-QMIX each have one limit-reaching member. Do not claim every model fully converged.

Putting the Modules Together: B Suddenly Slows Down

A teaching scenario showing module roles, not a timestamp-by-timestamp explanation extracted from a test trajectory.

01

Identify that not every unit is slow

Global features

Combine A/B buffers, unit WIP and operating states. The model may recognize A accumulation, B undersupply and assembly waiting for matching parts.

02

Distinguish a trend from a fluctuation

GRU

Review four frames: is B starvation persistent and A accumulation growing? The same present error can have different histories.

03

Coordinate for completed sets

QMIX

Training evaluates the joint actions with team value, allowing local scores to favor plant-wide balance instead of local utilization alone.

04

Wait for downstream consequences

3-step

Backpropagate actual rewards from several intervals so release decisions learn some delayed consequences sooner.

05

Revisit poorly understood turning points

PER

A large target-prediction gap increases the chance of replay. Priority changes again as prediction improves.

06

Combine independent training perspectives

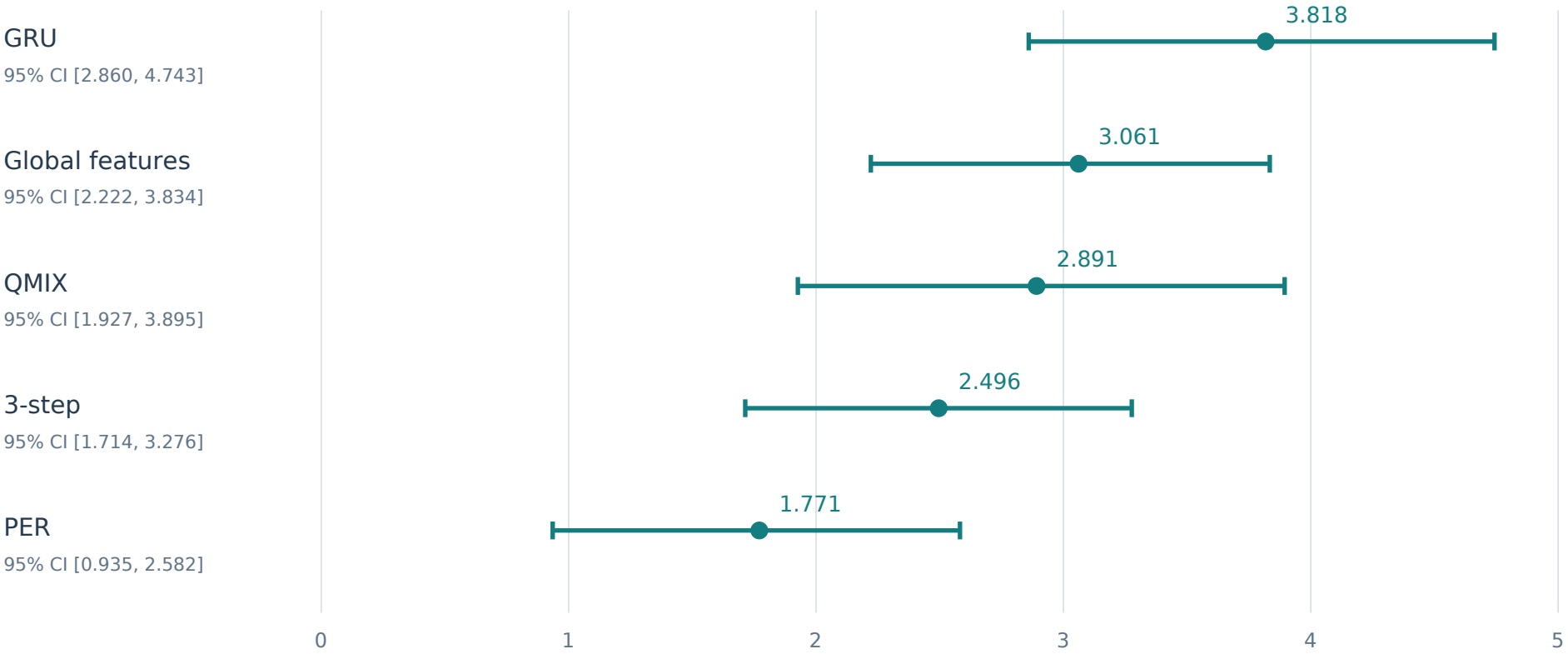
Q ensemble

At execution, five models score actions separately. Average the scores to choose the three unit actions, reducing accidental single-run preferences.

Ablations: How Much Reward Does Each Module Add?

PI stays off. Remove one module at a time and hold other settings fixed. All results here are test performance of five-model ensembles.

Reward advantage of retaining a module (full − ablated)



Horizontal bars: 95% intervals from 8,000 paired bootstrap resamples over shared test seeds; no multiplicity correction.

What does this figure support?

All five reward-difference intervals lie above zero. On this fixed test set, retaining each module improves mean reward.

What does it not support?

Do not add the five differences or claim statistically different contributions merely by ranking their means.

Better Reward Does Not Mean Every Metric Decreases

Aggregate reward trades off tracking, WIP and adjustment cost. NRMSE is not the only outcome.

Five-model strategy	Reward ×1000 ↑	NRMSE ↓	Mean WIP ↓	Cap-change points ↓
New M4 (all modules)	-35.540	0.12302	14.798	153.2
Without global features	-38.602	0.12285	15.853	321.1
Without GRU	-39.358	0.13515	14.561	275.8
Without QMIX	-38.432	0.12646	15.906	222.2
3-step → 1-step	-38.036	0.12416	15.798	273.4
PER → uniform replay	-37.311	0.12296	15.628	231.9

GRU has the clearest tracking contribution

Without GRU, NRMSE rises from 0.12302 to 0.13515, approximately +0.01214, with a paired 95% interval excluding zero. The other four NRMSE intervals include zero, so they should not all be described as significant tracking improvements.

Global features and PER affect more than tracking

Removing either slightly lowers mean NRMSE but increases inventory and adjustment points, worsening reward. This is consistent with improved cost trade-offs, but without reward-component decomposition it does not establish exact attribution.

Every row averages the same 30 test seeds. A change point is a decision where at least one cap changes, not Σ|cap change|.

Ablation Design and Evidence Limits

Mechanisms explain why a module may help. Ablations show what happens when it is removed from this configuration and test.

Leave-one-out experiments around new M4

The baseline includes global features, GRU, QMIX, 3-step, PER and five-model ensembling. Each ablation removes one component; PI is always off.

These are conditional contributions: GRU helps when the other modules remain. They are not results for adding GRU alone to original MADQN, nor a complete interaction study.

Single-model and ensemble evidence are different

Single-model summaries use 5 training members × 30 test seeds. An ensemble is one fixed team of those members, run on 30 seeds.

For single-model reward differences, global-feature and PER intervals include zero; GRU, QMIX and 3-step intervals exclude zero. Clear ensemble gains do not imply every single model benefits reliably.

What exactly changes when a module is removed?

No-GRU uses a current-frame feedforward network, not a parameter-count-matched substitute. No-QMIX uses separate per-agent TD learning.

Results compare these specific implementations, not abstract effects independent of capacity and learning targets.

Summary of supported findings

Under the specified environment and fixed training/test seeds, the PI-off M4 five-model independent Q ensemble achieves higher aggregate reward. Leave-one-module-out experiments show positive conditional contributions of all five modules to ensemble reward, with GRU providing the clearest tracking-accuracy benefit.

No new-seed confirmation in this round

Uses the fixed training and test seeds.

Test seeds have been used for version comparison, so this is not a fresh external final test. Intervals do not cover all uncertainty from environment changes or retraining multiple ensembles.

The Entire Algorithm on One Memory Card

See more → remember changes → coordinate → learn farther ahead → replay difficulties → combine models.

Term	Remember this
Global features	Let every unit see what the other two are doing.
GRU	Compress four frames into selective short-term memory.
QMIX	Mix local action scores into a team value during training.
3-step	Connect three real rewards to a successor-value estimate.
PER	Replay experiences the model has not yet learned accurately.
Independent Q ensemble	Five models score the same action; average before selecting.
PI OFF	Choose directly, without PI advice or residuals.

Key distinction: ensemble members are independent. Within each member, one GRU is shared and three dedicated Q heads remain. QMIX is training-only; global summaries are also used during execution.

Project evidence used in this guide

Architecture/training: run_controlled_m4_ablation.py. Networks, global summary and replay: run_madqn_ablation.py. Environment: pair_v2.py / baseline_algorithms.py.

Values come directly from m4_no_pi_ablation/comparison_summary.json, paired_contrasts.json and training_manifest.json. Only the test partition is used, not validation_audit.

Method references (clickable)

[1] Rashid et al., 2018: QMIX and monotonic value factorization.

[2] Schaul et al., 2016: Prioritized Experience Replay.

[3] van Hasselt et al., 2016: Double Q-learning.

[4] Cho et al., 2014: early gated recurrent unit architecture.

Status of illustrations and numbers

Code-based diagrams; illustrative examples; real ablations.